



Preliminary draft 22:14 19 May 2026

19 May 2026

Tanawin1701d@outlook.com

DFX4ML-ARCH (Progress Update 19-5-26)

Omitted for blind review

CERN, CH-1211 Geneva, Switzerland

Keywords: FPGA, Partial Reconfiguration, Dynamic Function Exchange, PYNQ, Xilinx

Summary

Contents

1	Architecture Upgrade	2
2	Barebone Testing	2
2.1	Hardware Setup	2
2.2	Session Setup	5
2.3	Execution Result	6
3	What's Next	8

1 Architecture Upgrade

A major refactor has been applied to almost all sections. The updated documentation is at [doc/tech_report_v0.2.pdf](#).

2 Barebone Testing

2.1 Hardware Setup

We test with two reconfigurable regions, each containing a single reconfigurable module. The configuration used for the build is shown below.

```
from lib.hw_build import HwBuildHelper
from lib.sw_build import SwBuildHelper

board_build_tcl = "./kv260/board_build.tcl"
constraint_xdc = "./kv260/constraint.xdc"

# Streamers (idx 0 = DMA)
dfx_streamers = [
    {"load_width": 4, "store_width": 4,
     "actual_width": 32, "amount_row": 1024}, # s0
    {"load_width": 4, "store_width": 4,
     "actual_width": 32, "amount_row": 1024}, # s1
]

# Regions: DMA -> r0 -> r1 -> DMA
dfx_regions = [
    {"load_streamers": [0], "store_streamers": [1]}, # r0
    {"load_streamers": [1], "store_streamers": [0]}, # r1
]

# RM schematics: rm_schematics[region][rm]
rm_schematics = [
    [{"load_io_map": [(0, 0)], "store_io_map": [(1, 0)]},], # r0 rm0
    [{"load_io_map": [(1, 0)], "store_io_map": [(0, 0)]},] # r1 rm0
]

hw_builder = HwBuildHelper(
    build_folder_path="./build_prj",
    dfx_root_path="../..",
    board="custom",
    board_build_tcl=board_build_tcl,
    constraint_xdc=constraint_xdc,
    user_repo_path="",
    user_rm_build_tcl_path="",
    req_gen_ip=1,
    num_core=4,
```

```

    clk_frq=99999001,           # Hz
    rm_index_width=2,          # max slots = 1 << width
    dfx_streamers=dfx_streamers,
    dfx_regions=dfx_regions,
    rm_schemetics=rm_schemetics,
    test_mode=1,
    vivado_path="/tools/Xilinx/Vivado/2023.2/bin/vivado",
    export_folder_path="./export"
)

hw_builder.run_build()
hw_builder.package_export_files()

sw_builder = SwBuildHelper(
    export_folder_path="./export",
    num_pr_region=2,
    rm_index_width=2,
    num_streamer=2,
)
sw_builder.package_export_file()

```

The resulting block design, built from the configuration listing above, is shown in Fig. 1. The internal hierarchies of `dfx_unified` and `dma_hier` are shown in Fig. 2 and Fig. 3, respectively.

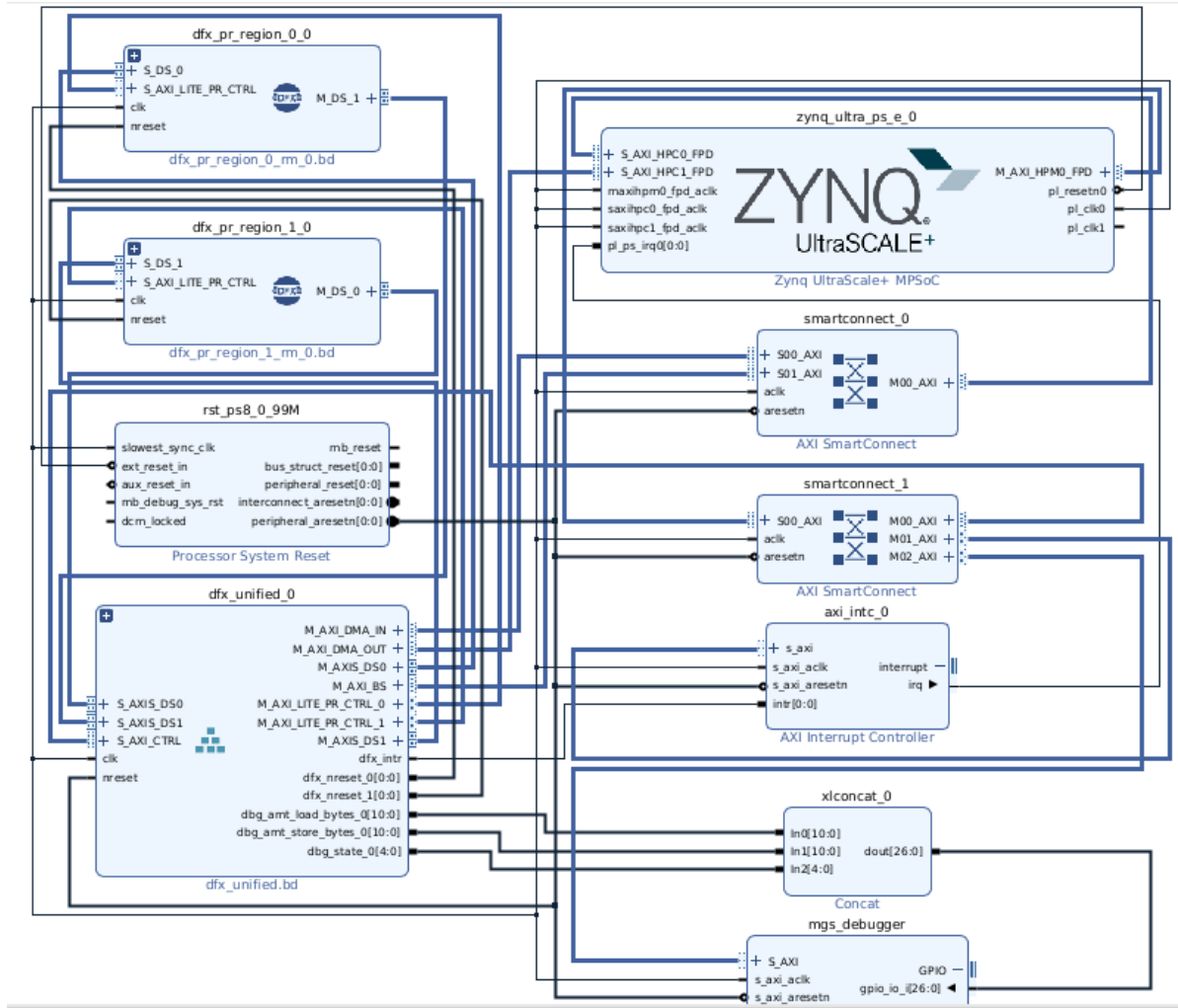


Figure 1: Block design of the multi-region test setup.

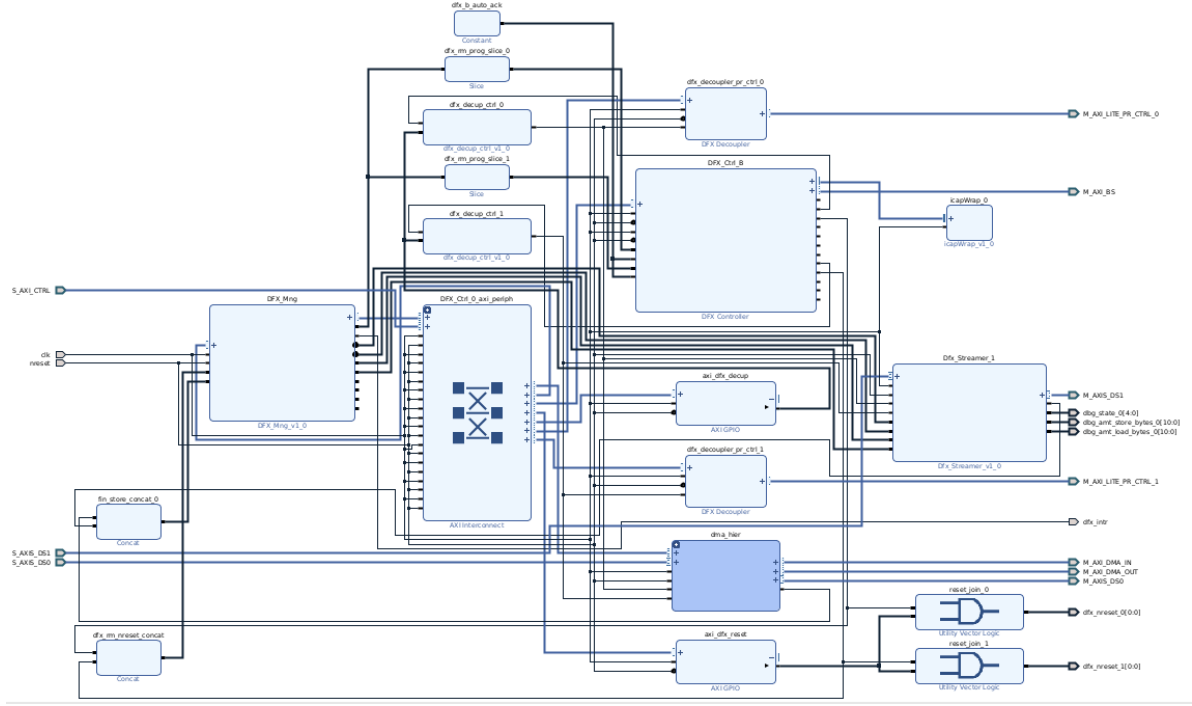


Figure 2: dfx_unified hierarchy.

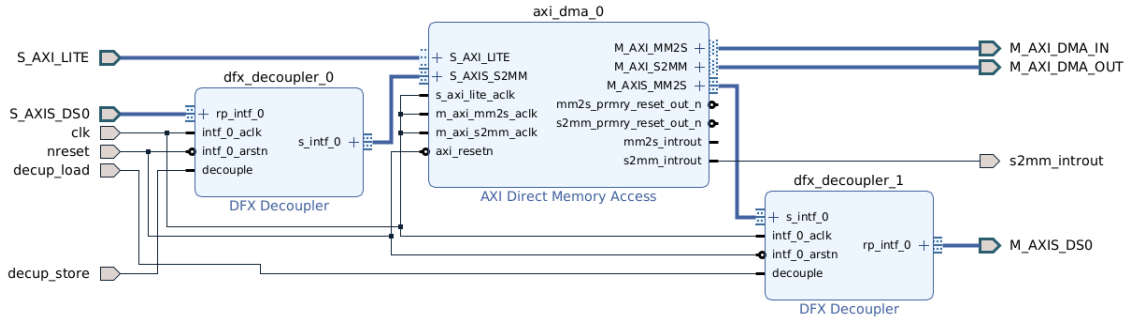


Figure 3: dma_hier hierarchy.

2.2 Session Setup

To orchestrate the flow, we use three sessions to support parallel execution:

- **First session:** reconfigure region 0.
- **Second session:** execute data on region 0 (DMA → magic streamer 1), while reconfiguring region 1.
- **Final session:** execute data on region 1 (magic streamer 1 → DMA).

```

# Slot 0: recon only
dfx_mng.set_whole_slot(0, [
    0,      0, # src (none)
    0,      0, # dst (none)
    0,      0, # profile counters
    0b01,   0b00, # RM select (recon) (exec) (RM 0)
    0,      0, # load mask (none), store mask (ch0)
    0,      # complete mask
    1,      # next_session -> slot 1
])

# Slot 1: DMA-load input, execute with RM 0, no store yet
dfx_mng.set_whole_slot(1, [
    buf_input_phya, buf_input_sz, # src
    0,      0, # dst (none)
    0,      0, # profile counters
    0b10, 0b01, # RM select for recon / exec (RM 0)
    0b01, 0b10, # load mask (ch0), store mask (none)
    0,      # complete mask
    2,      # next_session -> slot 2
])

# Slot 2: no load, execute with RM 0, DMA-store output -> wrap back
to slot 0
dfx_mng.set_whole_slot(2, [
    0,      0, # src (none)
    buf_out_phy, buf_out_sz, # dst
    0,      0, # profile counters
    0b00,   0b10, # RM select (RM 0)
    0b10,   0b01, # load mask (none), store mask (ch0)
    0,      # complete mask
    0,      # next_session -> slot 0 (wrap)
])

```

2.3 Execution Result

An example run of the three-session orchestration is shown in Fig. 4.

[cmd] shutdown the engine
[cmd] shutdown successfully

DFX Manager – Main Status	
MAIN_STATE	: SHUTDOWN
RECON_STATE	: SHUTDOWN
EXEC_STATE	: SHUTDOWN
LAST_SESSION	: 2
QUERY (cur/amt)	: 0/100
AMT_QUERY_PER_ITER	: 100
DMA_IP_ADDR	: 0xa0030000
PR_IP_ADDR	: 0xa0050000
INTERRUPT	: ena=1 status=1 (pending)

DFX Manager – Slot Data	
slot [0]	
src	: addr=0x0 size=0x0
des	: addr=0x0 size=0x0
recon	: rm_sel=0b1 prof=709983
exec	: rm_sel=0b0 prof=27
masks	: load=0b0 store=0b0 done=0b0
next	: slot 1
.....	
slot [1]	
src	: addr=0x2ac5190 size=0x190
des	: addr=0x0 size=0x0
recon	: rm_sel=0b10 prof=692862
exec	: rm_sel=0b1 prof=2747
masks	: load=0b1 store=0b10 done=0b10
next	: slot 2
.....	
slot [2]	
src	: addr=0x0 size=0x0
des	: addr=0x2acb190 size=0x190
recon	: rm_sel=0b0 prof=1
exec	: rm_sel=0b10 prof=237
masks	: load=0b10 store=0b1 done=0b1
next	: slot 0
.....	
slot [3]	
src	: addr=0x0 size=0x0
des	: addr=0x0 size=0x0
recon	: rm_sel=0b0 prof=0
exec	: rm_sel=0b0 prof=0
masks	: load=0b0 store=0b0 done=0b0
next	: slot 0

Figure 4: Example run of the three-session orchestration.

3 What's Next

1. We will perform further testing with dummy workloads, using multiple reconfigurable modules in each region.
2. We will also test the synthetic ML workload from HLS4ML into DFX4ML.